SECURITY AUDIT REPORT

TradeX Perpetual DEX

Trade Perpetuals on Pokémon Card Prices

tradex.cards

Prepared by calc1f4r

March 2025 · Final Report

Contents

1 About

calc1f4r is a smart contract security researcher specializing in EVM and non-EVM ecosystems (Solana, Sui, Aptos, Starknet). He is also experienced in auditing L1 nodes for Cosmos and Substrate-based chains. His work centers on improving blockchain security through rigorous code review, vulnerability research, and practical exploit analysis, with a focus on DeFi protocols.

Auditor contact: Available for follow-up engagements and remediation verification.

2 Disclaimer

A smart contract security review cannot find every vulnerability. It is limited by time, resources, and expertise. The goal is to catch as many issues as possible, but complete security cannot be guaranteed. To stay safer, it is best to conduct multiple reviews, run bug bounty programs, and keep monitoring on-chain activity.

3 Risk Classification

Findings are classified using a likelihood × impact matrix:

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

TradeX is a Solana-based perpetual futures DEX that enables users to trade perpetuals on Pokémon card prices. The protocol enables users to deposit collateral (USDC), open leveraged long/short positions on configurable markets, and settle PnL via an Ed25519-signed oracle price feed. Core flows include:

- **Deposit/Withdraw**: Users deposit USDC into a program-controlled vault; balances are tracked in `UserAccount` and `ExchangeState.total_deposits`.
- **Open/Close Position**: Users open positions with margin; PnL is settled on close using signed oracle prices.
- **Liquidation**: Under-collateralized positions can be liquidated; insurance fund covers shortfalls when vault liquidity is insufficient.
- **Funding**: Funding rates are updated periodically to balance long/short open interest.

The protocol uses PDAs for exchange state, vault, insurance fund, user accounts, and positions. Admin controls pause, market parameters, and fee withdrawal.

5 Audit Scope

Engagement duration: Approximately 8 days.

Scope: The entire `src/` directory was in scope for this security review. All production code paths were analyzed, including:

File	SLOC	Description
`src/perpdex.rs`	1,736	Main program logic — deposit, withdraw, open/close position, liquidate
`src/helpers.rs`	320	Oracle verification, funding rate calculation, PnL math
`src/oracle.rs`	93	Oracle state initialization and authority management
`src/state.rs`	243	Account structures — ExchangeState, UserAccount, Position, Market
`src/constants.rs`	34	Protocol constants and configuration
`src/error.rs`	32	Error definitions
`src/token_cpi.rs`	89	SPL Token CPI helpers
`src/lib.rs`	182	Program entrypoint and dispatch
Total	**2,729**	

6 Executive Summary

Over an **8-day engagement**, calc1f4r conducted a comprehensive security review of the TradeX Perpetual DEX smart contracts. The **entire** `src/` **codebase** was in scope. A total of **12 issues** were identified and confirmed, consisting of **1 High**, **5 Medium**, and **6 Low** severity findings. All findings have been addressed by the TradeX team.

6.1 Centralization and Trust Assumptions

- The exchange admin has privileged control over pause, market parameters, and fee withdrawal.
- Oracle authority signs prices off-chain; protocol trusts that key for settlement.
- Bootstrap flows (exchange and oracle init) can be front-run if not executed atomically by the intended operator.

6.2 Protocol Info

```
src/
├── perpdex.rs      # Main program, deposit/withdraw, open/close, liquidate
├── helpers.rs      # Oracle verification, funding, PnL calculation
├── oracle.rs       # Oracle state initialization
├── state.rs        # ExchangeState, UserAccount, Position, Market
├── constants.rs    # Protocol constants
├── error.rs
├── token_cpi.rs
└── lib.rs
```

Parameter	Value
Project	TradeX — Trade Perpetuals on Pokémon Card Prices
Repository	tradexcards/perp-program
Audit Commit	a5415fd
SLOC	2,729 (code only)
Audit Duration	~8 days
Scope	Full `src/` directory
Methodology	Manual code review, threat modeling
High Risk	1
Medium Risk	5
Low Risk	6
Total Issues	**12**

6.3 Issues Found

Title	Status
[H-01] Insurance Shortfall Payout Is Not Re-Credited To Closing User	Fixed
[M-01] `total_deposits` Semantic Mismatch Breaks Fee/Settlement Accounting	Fixed
[M-02] Exchange Bootstrap Can Be Front-Run To Capture Admin	Fixed
[M-03] PDA Prefunding Can Deny `CreateAccount`-Based Flows	Fixed
[M-04] Oracle Bootstrap Can Be Front-Run To Capture Price Authority	Fixed
[M-05] Illiquid Reference Price Drift Can Be Leveraged On-Chain	Fixed
[L-01] `insurance_balance` Can Drift From Real Insurance Token Balance	Fixed
[L-03] Funding Rate Calculation Truncates Small Imbalances To Zero	Fixed
[L-04] Delayed Funding Cranks Apply Only One Interval	Fixed
[L-05] `close_position` Does Not Bind `user_account` To Closing Signer	Fixed
[L-06] Ed25519 Scan Window Can Fail-Closed On Valid Complex Transactions	Fixed

6.4 Findings Summary

All findings were validated as reachable on current code paths, non-intentional, and written with conservative impact language. Each finding includes a **Fixed in** reference with the relevant commit hash for traceability.

7 Findings

7.1 High Risk

[H-01] Insurance Shortfall Payout Is Not Re-Credited To Closing User

Description: When `process_close_position` encounters vault shortfall, it caps user credit to vault liquidity and then uses insurance to refill the vault. The recovered insurance amount is not credited back to the same user in that close flow.

Vulnerability Details: At `perpdex.rs#L1076-L1078`, the payout is capped to `vault_balance`:

```
let vault_balance = read_token_amount(ctx.vault)?;
let actual_payout = payout.min(vault_balance);
let shortfall = payout.saturating_sub(actual_payout);
```

At `perpdex.rs#L1084-L1087`, the user is credited only with `actual_payout`:

```
ua.balance = ua
    .balance
    .checked_add(actual_payout)
    .ok_or(PerpDexError::MathOverflow)?;
```

Later, insurance covers shortfall at `perpdex.rs#L1134-L1150`, but there is no post-transfer user re-credit:

```
if shortfall > 0 {
    let insurance_balance = read_token_amount(ctx.insurance_fund)?;
    let transfer_amount = shortfall.min(insurance_balance);
    if transfer_amount > 0 {
        token_transfer_pda_signed(
            ctx.token_program,
            ctx.insurance_fund,
            ctx.vault,
            ctx.exchange_state,
            transfer_amount,
            &signer_seeds,
        )?;
    }

    let mut data = ctx.exchange_state.try_borrow_mut()?;
    let exchange = ExchangeState::load_mut(&mut data)?;
    exchange.insurance_balance = exchange.insurance_balance.saturating_sub(shortfall);
}
```

Proof of Concept:

1. Winning payout is `500`, vault has `300`, insurance has `200`.
2. Close computes `actual_payout = 300`, `shortfall = 200`.
3. User receives only `ua.balance += 300`.
4. Insurance transfers `200` into vault.
5. User never receives the missing `200`; that value remains unassigned in shared vault liquidity.

Impact: Users can be permanently under-credited on profitable closes when insurance is used to cover shortfall, even though insurance had enough funds.

Recommended Mitigation: After insurance transfer, credit the same user with `transfer_amount` and decrement insurance bookkeeping by the actual transferred amount:

```
if transfer_amount > 0 {
    token_transfer_pda_signed(/* ... */)?;

    let mut data = ctx.user_account.try_borrow_mut()?;
    let ua = UserAccount::load_mut(&mut data)?;
    ua.balance = ua
        .balance
        .checked_add(transfer_amount)
        .ok_or(PerpDexError::MathOverflow)?;
}
```

Fixed in: 22c52e8 — re-credit user with insurance-covered shortfall on close

---

7.2 Medium Risk

[M-01] `total_deposits` Semantic Mismatch Breaks Fee/ Settlement Accounting

Description: `ExchangeState.total_deposits` is updated as net deposits (`deposit +` , `withdraw -`), but `withdraw_fees(...)` also treats it as a hard vault-liability floor. This combines two different meanings into one variable. Because balance/fee state changes happen in other flows without updating `total_deposits` , protocol accounting can diverge across user balances, vault tokens, and fee-withdraw constraints.

Vulnerability Details:

`total_deposits` is only mutated in:

- `process_deposit`
- `process_withdraw`

```
exchange.total_deposits = exchange
    .total_deposits
    .checked_add(amount)
    .ok_or(PerpDexError::MathOverflow)?;

exchange.total_deposits = exchange
    .total_deposits
    .checked_sub(amount)
    .ok_or(PerpDexError::MathOverflow)?;
```

Fee accrual and payout flows do not keep that field aligned with liabilities:

- fees are accrued in `process_open_position` and `process_close_position` ,
- close credits user balance at `perpdex.rs#L1084-L1087` ,
- liquidation credits liquidator balance at `perpdex.rs#L1420-L1428` ,
- insurance can add vault tokens at `perpdex.rs#L1134-L1145` without changing `total_deposits` .

`withdraw_fees(...)` enforces:

`perpdex.rs#L2111-L2120`

```
if vault_balance.saturating_sub(amount) < exchange.total_deposits {
    return Err(PerpDexError::InsufficientBalance.into());
}
```

Proof of Concept:

1. Deposit `100` -> `vault=100` , `total_deposits=100` .

2. Trading accrues fees (`total_fees_collected > 0`) but does not reduce `total_deposits`.
3. Admin calls `withdraw_fees(1)`.
4. Check becomes `100 - 1 < 100` and reverts.

Additional reachable branch:

1. If insurance later tops up vault (`insurance -> vault`) without changing `total_deposits`, the same fee withdrawal may pass using that headroom.

Conditional settlement branch:

1. If some user balance grows above `total_deposits`, then `withdraw(amount)` can revert on `exchange.total_deposits.checked_sub(amount)`.

Impact: - Fee withdrawal can be blocked in normal steady-state accounting. - Settlement behavior becomes condition-dependent on unrelated vault inflows. - Accounting invariants become hard to reason about and monitor safely.

Recommended Mitigation:

Adopt a single coherent model:

1. keep `total_deposits` as informational net-deposit metric only, and
2. add explicit liability and fee-withdrawable accounting fields.

Alternative design:

- maintain a dedicated fee bucket/vault and withdraw protocol fees only from that bucket.

Fixed in: 4058fd1, b64185d, 638a69b — track `total_deposits` across all balance-changing flows (deposit, withdraw, open_position, close_position, liquidate, add_margin)

[M-02] Exchange Bootstrap Can Be Front-Run To Capture Admin

Description: `process_initialize_exchange` creates the core PDAs and sets the first admin. The initializer-supplied signer is written directly into `exchange.admin` during bootstrap. There is no fixed bootstrap-authority check in this path, so the first caller can capture admin if initialization is not performed atomically by the intended operator.

Vulnerability Details:

At `perpdex.rs#L253-L260`, the exchange state PDA is created by whichever signer calls init first:

```
CreateAccount {
    from: ctx.admin,
    to: ctx.exchange_state,
    lamports: rent_exempt_minimum(ExchangeState::ACCOUNT_SIZE),
    space: ExchangeState::ACCOUNT_SIZE as u64,
    owner: program_id,
}
.invoke_signed(&[exchange_signer])?;
```

At `perpdex.rs#L321-L321`, admin authority is set from caller:

```
exchange.admin = *ctx.admin.address().as_array();
```

Once set, admin-gated handlers trust that stored key (for example, `process_set_paused` and `process_update_market_params`).

Proof of Concept:

1. Program is deployed, but exchange bootstrap has not been executed.
2. Attacker computes deterministic PDA addresses from public seeds.
3. Attacker calls `initialize_exchange` first with attacker signer as `admin`.
4. Program initializes PDAs and stores attacker key as `exchange.admin`.
5. Intended operator's later initialization attempt fails because state already exists.
6. Attacker controls admin-only operations until key rotation.

Impact: A successful front-run at bootstrap can grant persistent control over admin operations, including pause/unpause, market parameter changes, and fee control.

Recommended Mitigation:

Add explicit bootstrap authorization:

- hardcode expected bootstrap authority, or
- verify against immutable deploy-time config, or
- enforce deployment+initialization in one controlled transaction sequence.

Fixed in: c1d4782 — restrict exchange initialization to bootstrap authority

---

[M-03] PDA Prefunding Can Deny `CreateAccount`-Based Flows

Description: The program creates deterministic PDAs through direct `CreateAccount` calls. If an attacker pre-funds a target PDA address first, the later protocol create operation can fail because the destination is already in use. This is a denial-of-service pattern on deterministic seeds, especially for user onboarding and per-position PDA creation.

Vulnerability Details:

PDA derivation is deterministic and public (for example, via `find_pda`).

Direct create paths include:

- exchange bootstrap create path in `process_initialize_exchange`,
- user account creation in `process_create_user_account`,
- position creation in `process_open_position`.

Representative logic:

```
CreateAccount {
    from: ctx.owner,
    to: ctx.user_account,
    lamports: rent_exempt_minimum(UserAccount::ACCOUNT_SIZE),
    space: UserAccount::ACCOUNT_SIZE as u64,
    owner: program_id,
}
.invoke_signed(&[ua_signer])?;
```

Proof of Concept:

1. Attacker computes a target PDA from known seeds (for example: `USER_ACCOUNT_SEED || victim_pubkey`).
2. Attacker pre-funds that PDA address with lamports.
3. Victim calls `create_user_account`.
4. Program reaches `CreateAccount` for already-used destination.
5. Transaction fails and victim cannot initialize account through normal path.

The same grief pattern applies to targeted `POSITION_SEED` tuples.

Impact: Attackers can block selected account-creation flows for specific users/markets, causing onboarding and trading disruption without direct vault theft.

Recommended Mitigation:

Use resilient initialization for pre-funded system-owned addresses:

1. verify expected PDA,
2. top-up rent if needed,

3. `allocate` and `assign` via signed CPI instead of relying exclusively on one-step `CreateAccount`.

Fixed in: c7afa04 — use resilient PDA creation to prevent prefunding DoS

[M-04] Oracle Bootstrap Can Be Front-Run To Capture Price Authority

Description: `process_initialize_oracle` creates `oracle_state` and sets `oracle_state.authority` directly from the initializer signer. If initialization is not performed atomically by the intended operator, the first caller can capture the oracle signing authority used by price verification.

Vulnerability Details:

At `oracle.rs#L67-L81`:

```
CreateAccount {
    from: ctx.authority,
    to: ctx.oracle_state,
    lamports: rent_exempt_minimum(OracleState::ACCOUNT_SIZE),
    space: OracleState::ACCOUNT_SIZE as u64,
    owner: program_id,
}
.invoke_signed(&[oracle_signer])?;

let oracle_state = OracleState::init(&mut data)?;
oracle_state.authority = *ctx.authority.address().as_array();
```

Later, signed price verification trusts that stored key:

`helpers.rs#L125-L130`

```
let data = oracle_state.try_borrow()?;
let os = OracleState::load(&data)?;
let expected_pubkey = os.authority;
```

Proof of Concept:

1. Program is deployed, `oracle_state` is not initialized.
2. Attacker computes oracle PDA from `ORACLE_SEED`.
3. Attacker calls `initialize_oracle` first with attacker signer.
4. Program stores attacker key as `oracle_state.authority`.
5. Subsequent `verify_signed_price` accepts attacker-signed prices.

Impact: Captured oracle authority can control accepted signed prices for open/close/liquidation flows, creating protocol-wide pricing integrity risk.

Recommended Mitigation:

Restrict bootstrap with explicit authorization:

- hardcode/immutably configure expected initializer key, or
- enforce deploy-and-initialize as one controlled rollout step.

Fixed in: c1d4782 — restrict oracle initialization to bootstrap authority

---

[M-05] Illiquid Reference Price Drift Can Be Leveraged On-Chain

Description: The protocol accepts fresh Ed25519-signed prices and uses them directly for PnL settlement in open/close/liquidation paths. On-chain checks validate signature authenticity and timestamp freshness, but they do not validate source market quality (liquidity/depth) or abnormal drift. If the off-chain oracle source is economically manipulable (for example, illiquid 7-day averaged card prices), valid signed updates can still carry manipulated economics into settlement.

Vulnerability Details:

Price validation logic in `verify_signed_price` focuses on signature and staleness:

```
if price == 0 {
    return Err(PerpDexError::InvalidOraclePrice.into());
}

let age = clock.unix_timestamp
    .checked_sub(timestamp)
    .ok_or(PerpDexError::MathOverflow)?;
if age > MAX_ORACLE_PRICE_AGE || age < -30 {
    return Err(PerpDexError::OraclePriceStale.into());
}
```

Trading flows then trust `price` directly:

- open: `perpdex.rs#L822-L831`
- close: `perpdex.rs#L1011-L1020`
- liquidate: `perpdex.rs#L1317-L1326`

```
verify_signed_price(..., product_id, price, timestamp, 0)?;
let oracle_price = price;
```

Proof of Concept:

1. Attacker opens long exposure using valid signed price.
2. Attacker nudges an illiquid source market upward over several days.
3. Oracle backend signs the drifted (but source-derived) price.
4. Attacker closes at higher signed price and realizes leveraged PnL.
5. Protocol settles from vault/insurance despite source-level manipulation.

Example (within current limits):

- per-account size cap is `MAX_POSITION_SIZE_PER_USER` (`100,000`), so attacker can aggregate across multiple wallets.

Impact: Manipulated source prints can be converted into leveraged protocol payouts, creating insolvency pressure when notional size is large versus true market liquidity.

Recommended Mitigation:

Harden oracle architecture:

1. use multi-source robust aggregation (median/trimmed mean),
2. enforce minimum depth/volume filters off-chain,
3. add on-chain deviation/circuit-breaker guards,
4. apply stricter leverage/OI limits for illiquid products.

Fixed in: 9b1ceaa — add on-chain price deviation circuit breaker with market migration

---

7.3 Low Risk

[L-01] `insurance_balance` Can Drift From Real Insurance Token Balance

Description: Liquidation flows cap actual token movement to available vault/insurance balances, but accounting sometimes applies uncapped amounts to `exchange.insurance_balance`. This can desynchronize protocol bookkeeping from actual insurance-fund token balance.

Vulnerability Details:

In `process_liquidate`, transfer is capped:

`perpdex.rs#L1437-L1450`

```
let vault_balance = read_token_amount(ctx.vault)?;
let capped_fee = insurance_fee.min(vault_balance);
if capped_fee > 0 {
    token_transfer_pda_signed(..., capped_fee, ...)?;
}
```

But accounting uses uncapped `insurance_fee`:

`perpdex.rs#L1476-L1479`

```
exchange.insurance_balance = exchange
    .insurance_balance
    .checked_add(insurance_fee)
    .ok_or(PerpDexError::MathOverflow)?;
```

The same pattern exists in keeper liquidation:

- capped transfer: `perpdex.rs#L1683-L1696`
- uncapped accounting add: `perpdex.rs#L1722-L1725`

Proof of Concept:

1. Liquidation computes `insurance_fee = 200`, vault holds `100`.
2. Transfer sends only `100` to insurance fund due to capping.
3. Accounting adds full `200` to `exchange.insurance_balance`.
4. Bookkeeping overstates insurance by `100`.

Impact: Insurance accounting can become inaccurate, weakening operator/off-chain risk visibility and potentially affecting future logic that assumes this field is exact.

Recommended Mitigation:

Always update bookkeeping with actual transferred amounts:

```
exchange.insurance_balance = exchange
    .insurance_balance
    .checked_add(capped_fee)
    .ok_or(PerpDexError::MathOverflow)?;
```

Also apply the same rule for subtraction paths (use actual debited amount).

Fixed in: 07b7a4a — use actual transferred amounts for insurance bookkeeping

---

[L-03] Funding Rate Calculation Truncates Small Imbalances To Zero

Description: `process_update_funding_rate` scales by `BPS_DENOMINATOR` and then divides by the same denominator before storing the rate. Under integer arithmetic, this truncates sub-1-bps results to `0`.

Vulnerability Details:

At `perpdex.rs#L1836-L1847`:

```
let rate = imbalance
    .checked_mul(BPS_DENOMINATOR as i128)
    .ok_or(PerpDexError::MathOverflow)?
    .checked_mul(FUNDING_RATE_CAP_BPS as i128)
    .ok_or(PerpDexError::MathOverflow)?
    .checked_div(total_oi as i128)
    .ok_or(PerpDexError::MathOverflow)?
    .checked_div(BPS_DENOMINATOR as i128)
    .ok_or(PerpDexError::MathOverflow)?;
```

Then accumulator updates use that truncated value:

`perpdex.rs#L1849-L1857`

```
market.cumulative_funding_rate_long = market
    .cumulative_funding_rate_long
    .checked_add(funding_rate)
    .ok_or(PerpDexError::MathOverflow)?;
```

Proof of Concept:

1. `total_oi = 10,000,000`, imbalance `= 1,900,000`, cap `= 5` bps.
2. Effective rate expression is `(imbalance * 5) / total_oi = 0.95`.
3. Integer truncation stores `0`.
4. Funding update applies no correction for that interval.

Impact: Small-to-moderate OI skews can persist longer than intended because funding pressure is quantized to zero in those ranges.

Recommended Mitigation:

Keep higher internal precision for funding accumulators and downscale only when computing per-position payment in `calculate_funding_payment`.

Fixed in: 0752b41 — preserve sub-bps precision in funding rate accumulators

---

[L-04] Delayed Funding Cranks Apply Only One Interval

Description: `process_update_funding_rate` checks that enough time has elapsed but applies funding exactly once, even if multiple funding intervals were missed. The function then sets `last_funding_time` directly to `now`, which skips intermediate accrual windows.

Vulnerability Details:

Elapsed-time gate:

`perpdex.rs#L1815-L1821`

```
let time_elapsed = clock
    .unix_timestamp
    .checked_sub(market.last_funding_time)
    .ok_or(PerpDexError::MathOverflow)?;
if time_elapsed < FUNDING_INTERVAL {
    return Err(PerpDexError::FundingTooEarly.into());
}
```

Single-interval update + direct timestamp jump:

`perpdex.rs#L1849-L1857`

```
market.cumulative_funding_rate_long = market
    .cumulative_funding_rate_long
    .checked_add(funding_rate)
    .ok_or(PerpDexError::MathOverflow)?;
market.cumulative_funding_rate_short = market
    .cumulative_funding_rate_short
    .checked_sub(funding_rate)
    .ok_or(PerpDexError::MathOverflow)?;
market.last_funding_time = clock.unix_timestamp;
```

Proof of Concept:

1. Funding interval is 8h.
2. Crank is delayed 24h.
3. Function applies one `funding_rate` update.
4. `last_funding_time` jumps to current time.
5. Two missed intervals are not accrued.

Impact: Delayed cranks can under-accrue funding, causing fairness drift between long and short sides and weakening expected funding mechanics.

Recommended Mitigation:

Scale rate updates by elapsed period count and advance `last_funding_time` by `periods * FUNDING_INTERVAL`, not directly to `now`.

Fixed in: ec4fb18 — apply funding for all missed intervals in delayed cranks

---

[L-05] `close_position` Does Not Bind `user_account` To Closing Signer

Description: `process_close_position` verifies position ownership but does not explicitly verify that the provided `user_account` belongs to the same signer. Because `ClosePositionAccounts.user_account` is declared as generic `account_mut`, the handler mutates that account without a direct `ua.owner` ownership assertion in this path.

Vulnerability Details:

Account context declaration:

`perpdex.rs#L113-L124`

```
pub struct ClosePositionAccounts<'a> {
    pub user_account:         account_mut,
    pub position:             account_mut,
    // ...
    pub owner:                signer_mut,
}
```

Position owner is checked:

`perpdex.rs#L1045-L1047`

```
if pos_owner != *ctx.owner.address().as_array() {
    return Err(PerpDexError::Unauthorized.into());
}
```

But `user_account` is then mutated without `ua.owner == signer` check:

`perpdex.rs#L1082-L1099`

```
let mut data = ctx.user_account.try_borrow_mut()?;
let ua = UserAccount::load_mut(&mut data)?;
ua.balance = ua.balance.checked_add(actual_payout).ok_or(PerpDexError::MathOverflow)?;
ua.total_realized_pnl = ua.total_realized_pnl.checked_add(net_pnl).ok_or(PerpDexError::MathOverflow)?;
ua.total_fees_paid = ua.total_fees_paid.checked_add(fee).ok_or(PerpDexError::MathOverflow)?;
ua.open_position_count = ua.open_position_count.checked_sub(1).ok_or(PerpDexError::MathOverflow)?;
```

Proof of Concept:

1. Attacker owns a valid position and can call `close_position`.
2. Attacker supplies another program-owned `UserAccount` as `ctx.user_account`.

3. Position ownership check still passes (it validates only `position.owner`).
4. Handler mutates supplied `user_account` fields.
5. Victim accounting can be altered if victim state satisfies arithmetic checks.

Impact: State/accounting integrity can be violated by cross-account mutation in close flow, creating griefing and incorrect accounting outcomes.

Recommended Mitigation:

In `process_close_position`, explicitly enforce:

```
let ua = UserAccount::load(&data)?;
if ua.owner != *ctx.owner.address().as_array() {
    return Err(PerpDexError::Unauthorized.into());
}
```

For stronger binding, also verify canonical user-account PDA for signer.

Fixed in: 8f8eed8 — bind user_account to closing signer in close_position

---

[L-06] Ed25519 Scan Window Can Fail-Closed On Valid Complex Transactions

Description: `verify_signed_price` scans only the first 8 instructions for Ed25519 verification and may early-exit on an oracle-key message mismatch. This does not bypass signature checks; it fail-closes by rejecting valid transactions in some composed layouts.

Vulnerability Details:

Bounded scan window:

`helpers.rs#L141-L149`

```
let max_scan: usize = 8;
for idx in 0..max_scan {
    let ix = match instructions.load_instruction_at(idx) {
        Ok(ix) => ix,
        Err(_) => break,
    };
    // ...
}
```

Early abort on first oracle-key mismatch:

`helpers.rs#L199-L201`

```
if &ix_data[msg_offset..msg_offset + msg_size] != &expected_msg {
    return Err(PerpDexError::InvalidSignedPrice.into());
}
```

Proof of Concept:

1. Transaction contains valid Ed25519 proof at index `>= 8`.
2. Program scans only `0..7`.
3. Valid proof is never considered.
4. Instruction fails with `InvalidEd25519Instruction`.

Alternative branch:

1. Two Ed25519 instructions are present with same oracle pubkey.
2. Earlier one has non-matching message.
3. Function returns `InvalidSignedPrice` before reaching later valid proof.

Impact: False rejects reduce composability and availability for bundled transactions, but do not create unauthorized price-acceptance risk.

Recommended Mitigation:

- scan all available instructions (or larger safe cap),
- continue scanning after non-matching messages,
- fail only after full scan confirms no valid matching proof.

Fixed in: 9b5434b — widen Ed25519 scan window and continue on message mismatch

---

Report prepared by calc1f4r

March 2025 · TradeX Perpetual DEX Security Audit · tradex.cards